

JOIN

JOIN залепва 2 таблици 2 за друга, като обикновения JOIN е INNER JOIN по подразбиране, дори ако INNER е пропусната да се изпише. Залепването се осъществява чрез връзката между ключовете id.

1 зад: Искаме от БД geography да изпишем имената на столиците, в кои държави и континенти се намират.

Решение: За целта ни трябва 2 таблици:

Countries:

country_code	iso_code	country_name	currency_code	continent_code	population	are_in_sq_km	capital
AD	AND	Andorra	EUR	EU	84000	468	Andorra la Vella
AE	ARE	United Arab Emirates	AED	AS	4975593	82880	Abu Dhabi
AF	AFG	Afghanistan	AFN	AS	29121286	647500	Kabul
AG	ATG	Antigua and Barbuda	XCD	NA	86754	443	St. John's
AI	AIA	Anguilla	XCD	NA	13254	102	The Valley
AL	ALB	Albania	ALL	EU	2986952	28748	Tirana
...	...	...	...	...	...	...	...

Continents:

continent_code	continent_name
AF	Africa
AN	Antarctica
AS	Asia
EU	Europe
LM	Lemuria
NA	North America
...	...

Като връзката между двете таблици няма да е с id, а в случая е с continent_code.

Спазваме синтаксиса за JOIN:

SELECT колоните от двете таблици, които ни трябва

FROM таблица1

JOIN

таблица2

ON връзката таблица1.id = таблица2.id;

-- Столица, държава, континент

```
SELECT capital AS Capital, country_name AS Country, continent_name AS Continent  
FROM countries
```

```
JOIN
```

```
continents
```

```
ON countries.continent_code = continents.continent_code;
```

t1

t2

Резултат в 250 реда, като обръщаме внимание, че са подредени по азбучен ред на таблица2 continents:

Capital	Country	Continent
Luanda	Angola	Africa
Ouagadougou	Burkina Faso	Africa
Bujumbura	Burundi	Africa
Porto-Novo	Benin	Africa
Gaborone	Botswana	Africa
Kinshasa	Democratic Republic of the Congo	Africa

✓ 150 10:36:42 SELECT capital AS Capital, country_name AS Country, continent_name AS Continent FR... 250 row(s) returned

За да има яснота, обаче, в практиката е наложено да се използват псевдоними на таблиците. В случая, аз съм изобразила с t1 и t2 таблица1 и таблица2.

По принцип, за имена на псевдонимите се избира първата буква от таблицата, например peaks AS p, mountains AS m, обаче в случая имаме 2 еднакви букви countries и continents, така че едната се кръщава с a, другата с b:

```
-- добавяме псевдоними за таблиците
SELECT capital AS Capital, country_name AS Country, continent_name AS Continent
FROM countries AS a
JOIN
continents AS b
ON a.continent_code = b.continent_code;
```

Този код е абсолютно същият, връща същият резултат от 250 реда, просто са използвани псевдоними вместо таблици.

За още по-голяма яснота, се указва и псевдоним коя колона от коя таблица е:

```
-- псевдоними за таблиците + колоните
SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Continent
FROM countries AS a
JOIN
continents AS b
ON a.continent_code = b.continent_code;
```

Извод: в секция SELECT може да се изпишат колоните без псевдонима на таблицата, но щом веднъж е въведен, от него надолу кодът задължително трябва да уточнява коя колона от коя таблица е посредством псевдонима.

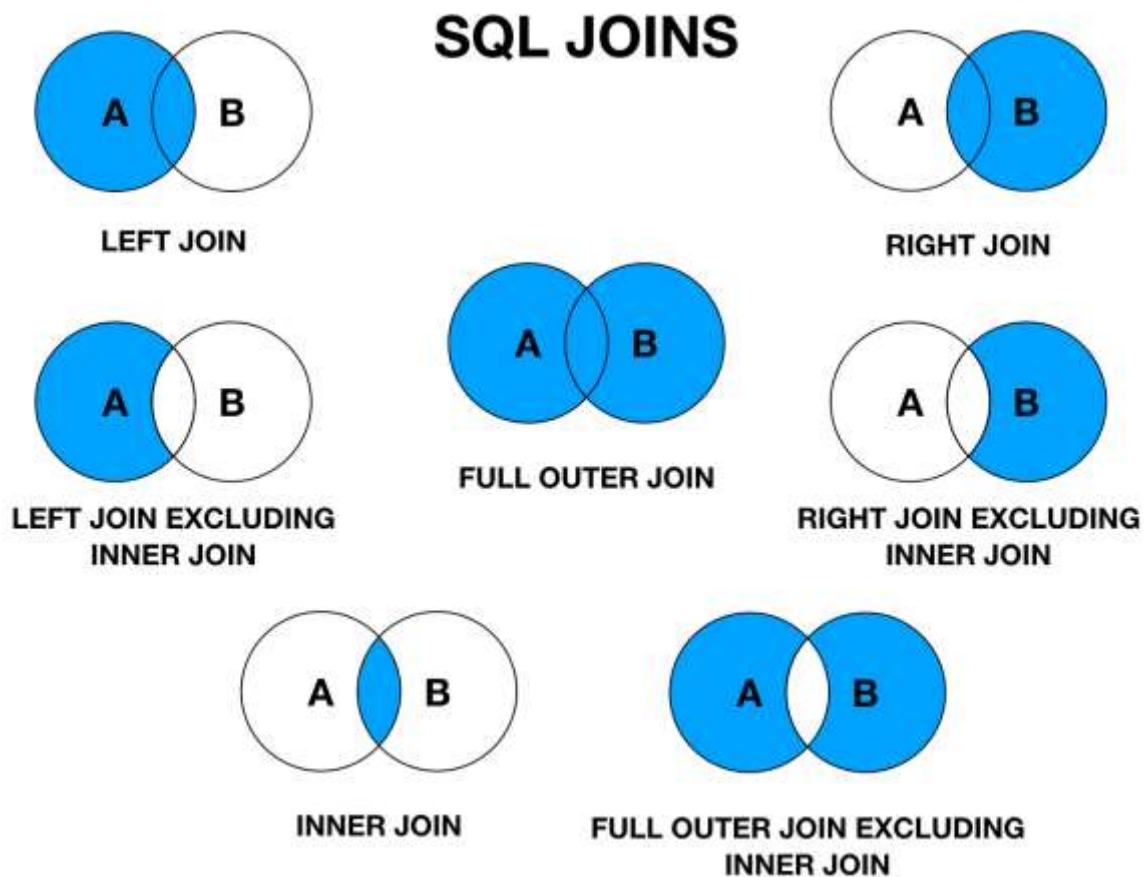
По този начин, съединяването на 3 и повече таблици става много по-разбираемо, при това не се налага да се изписват пълните имена.

Има и друг вид синтаксис, който се среща без AS, но принципно е по-желателно да се изписва по конвенция:

-- псевдоними за таблиците БЕЗ AS

```
SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Continent
FROM countries(a) -- AS
JOIN continents(b) -- AS
ON a.continent_code = b.continent_code;
```

Освен INNER JOIN, който връща съвпадащите записи, има още няколко вида:



INNER JOIN (просто JOIN – съвпадащите записи). Връзка само с вътрешните записи - връщаща само редовете, отговарящи на условието за свързване.

LEFT (или RIGHT) OUTER JOIN (връзка и със записите, които са външни отляво / отдясно)
Връща резултата на връзката с вътрешните записи и също така несъвпадащите записи от лявата (или дясната) таблица.

FULL OUTER JOIN (просто FULL JOIN – всички записи от двете, обединява LEFT JOIN + RIGHT JOIN). Връзка с всички външни записи)
Връща резултата на INNER JOIN и всички несъвпадащи записи.

CROSS JOIN комбинира всеки ред от първата таблица с всеки ред от втората, кръстосани всяко с всяко – Декартово произведение.

Ако вземем същата задача, но вместо JOIN или INNER JOIN, които връщат 250 записа (колкото са държавите) напишем RIGHT JOIN

```
-- псевдоними за таблиците + колоните
SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Continent
FROM countries AS a
RIGHT JOIN
continents AS b
ON a.continent_code = b.continent_code;
```

Резултатът е 252 реда:

✓ 171 11:00:07 SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Contine... 252 row(s) returned

Т.е. 2 реда повече. Откъде идват?

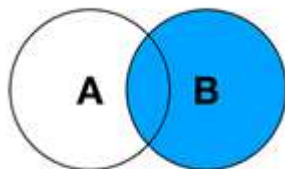
Причината е, че в таблица Continents сме въвели и стоят 2 измислени континента – Lemuria и Pangea, в които няма държави:

continent_code	continent_name
AS	Asia
EU	Europe
LM	Lemuria
NA	North America
OC	Oceania
PG	Pangea
SA	South America
NULL	NULL

Ще използваме филтъра за проверка в резултата на нашата заявка:

Capital	Country	Continent
NULL	NULL	Lemuria

Capital	Country	Continent
NULL	NULL	Pangea



RIGHT JOIN

Виждаме, че колоните от таблица1 countries са празни, а от таблица2 continents имаме 2 записа, т.е. това, което RIGHT JOIN е направило, е да вземе съвпадащите записи от лявата таблица и ЦЯЛАТА (ВЪНШНА) дясна. При това, резултатът е подреден по азбучен ред на дясната таблица (Africa). RIGHT JOIN показва, че дясната таблица е по-важна и подреждането е по нея.

13 RIGHT JOIN

Result Grid			
	Capital	Country	Continent
▶	Luanda	Angola	Africa
	Ouagadougou	Burkina Faso	Africa
	Bujumbura	Burundi	Africa
	Porto-Novo	Benin	Africa

-- псевдоними за таблиците + колоните

```
SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Continent
FROM countries AS a
```

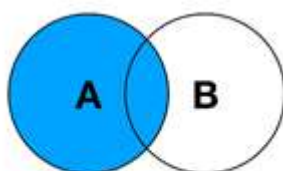
LEFT JOIN

```
continents AS b
```

```
ON a.continent_code = b.continent_code;
```

Резултатът е 250 реда, но този път подредбата е различна:

Capital	Country	Continent
Andorra la Vella	Andorra	Europe
Abu Dhabi	United Arab Emirates	Asia
Kabul	Afghanistan	Asia
St. John's	Antigua and Barbuda	North America
The Valley	Anguilla	North America
Tirana	Albania	Europe



LEFT JOIN

Те са подредени както е взета цялата ЛЯВА таблица 1:1 и само съвпадащите записи от дясната. Виждат се, разбира се, само онези колони, които сме избрали.

✓ 174 11:21:14 SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Contine... 250 row(s) returned

country_code	iso_code	country_name	currency_code	continent_code	population	are_in_sq_km	capital
AD	AND	Andorra	EUR	EU	84000	468	Andorra la Vella
AE	ARE	United Arab Emirates	AED	AS	4975593	82880	Abu Dhabi
AF	AFG	Afghanistan	AFN	AS	29121286	647500	Kabul
AG	ATG	Antigua and Barbuda	XCD	NA	86754	443	St. John's
AI	AIA	Anguilla	XCD	NA	13254	102	The Valley

Нека сега изобщо махнем JOIN и видим допустим ли е запис, подобен на

```
SELECT * колона1, колона2, колона3
FROM таблица1
```

JOIN

таблица2;

-- Декартово произведение - без ON връзка

```
SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Continent
FROM countries AS a
JOIN
continents AS b;
```

Без връзка ON връща над 1000 резултата, които WorkBench не може да изобрази, затова връща само 1000, но реално за всички записи от първата таблица по всички от втората, което наричаме Декартово произведение. Неговият изход е $250 * 9$ в моя случай, защото то не се интересува дали дадената държава се намира в дадения континент, а просто ги умножава всяко с всяко:

Capital	Country	Continent
Andorra la Vella	Andorra	South America
Andorra la Vella	Andorra	Pangea
Andorra la Vella	Andorra	Oceania
Andorra la Vella	Andorra	North America
Andorra la Vella	Andorra	Lemuria
Andorra la Vella	Andorra	Europe
Andorra la Vella	Andorra	Asia
Andorra la Vella	Andorra	Antarctica
Andorra la Vella	Andorra	Africa
Abu Dhabi	United Arab Emirates	South America
Abu Dhabi	United Arab Emirates	Pangea
Abu Dhabi	United Arab Emirates	Oceania

✓ 275 14:00:50 SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Contine... 1000 row(s) returned

В този случай дори може да се изпусне думата JOIN:

```
SELECT * колона1, колона2, колона3
FROM таблица1, таблица2;
```

-- Декартово произведение - без JOIN, без ON връзка

```
SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Continent
FROM countries AS a, continents AS b;
```

Резултат:

Capital	Country	Continent
Andorra la Vella	Andorra	South America
Andorra la Vella	Andorra	Pangea
Andorra la Vella	Andorra	Oceania
Andorra la Vella	Andorra	North America
Andorra la Vella	Andorra	Lemuria
Andorra la Vella	Andorra	Europe

✓ 176 11:34:33 SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Contine... 1000 row(s) returned

Този запис е допустим – таблица1 и таблица2 са разделени само със запетая, но резултатът е 1000 реда, което е лимита. Всъщност, истинският резултат е всички записи от първата таблица * всички от втората. За 1 ред от първата са изредени всички от втората, после същото се повтаря и с втория ред от първата таблица и така до последно, общо за 250 реда от таблица1 и 9 от таблица 2, Декартово произведение $250 * 9$, но SQL максимумът е 1000 реда. Декартовото произведение е кръстосаното слепване CROSS JOIN.

```
SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Continent
FROM countries AS a
CROSS JOIN continents AS b;
```

Нека вече добавим условие – Всички столици на държави, които се намират в Европа:

```
-- JOIN + WHERE за Европа
SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Continent
FROM countries AS a
JOIN
continents AS b
ON a.continent_code = b.continent_code
WHERE (b).continent_code = 'EU'; Условие + в коя таблица
```

Резултат в 53 реда Европейски държави:

Capital	Country	Continent
Andorra la Vella	Andorra	Europe
Tirana	Albania	Europe
Vienna	Austria	Europe
Mariehamn	Åland	Europe
Sarajevo	Bosnia and Herzegovina	Europe
Brussels	Belgium	Europe

✓ 179 11:43:58 SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Contine... 53 row(s) returned

Можем да направим и подредба по азбучен ред (ASC по подразбиране):

```
-- JOIN + WHERE + ORDER BY за Европа
SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Continent
FROM countries AS a
JOIN
continents AS b
ON a.continent_code = b.continent_code
WHERE (b).continent_code = 'EU'
ORDER BY (a).capital;
```

Отново указваме от коя точно таблица.

Capital	Country	Continent
Amsterdam	Netherlands	Europe
Andorra la Vella	Andorra	Europe
Athens	Greece	Europe
Belgrade	Serbia	Europe
Berlin	Germany	Europe
Berne	Switzerland	Europe

Последно – може да ограничим и резултатите с LIMIT:

```
-- JOIN + WHERE + ORDER BY + LIMIT за Европа
SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Continent
FROM countries AS a
JOIN
continents AS b
ON a.continent_code = b.continent_code
WHERE b.continent_code = 'EU'
ORDER BY a.capital
LIMIT 5;
```

✓ 181 11:51:13 SELECT a.capital AS Capital, a.country_name AS Country, b.continent_name AS Contine... 5 row(s) returned

Нека намерим коя река през коя държава минава. За целта ни трябва 3 таблици:

rivers, междинната с двата външни ключа countries_rivers, която осъществява връзката n:n = много към много и таблица countries:

rivers AS r					
id	river_name	length	drainage_area	average_discharge	outflow
1	Nile	6650	3254555	5100	Mediterranean
2	Amazon	6400	7050000	219000	Atlantic Ocean
3	Yangtze	6300	1800000	31900	East China Sea
4	Mississippi	6275	2980000	16200	Gulf of Mexico
5	Yenisei	5539	2580000	19600	Kara Sea

countries_rivers AS cr	
river_id	country_code
1	BI
1	CD
1	EG
1	ER
1	ET

n: n = МНОГО : МНОГО

countries AS c							
country_code	iso_code	country_name	currency_code	continent_code	population	are_in_sq_km	capital
AD	AND	Andorra	EUR	EU	84000	468	Andorra la Vella
AE	ARE	United Arab Emirates	AED	AS	4975593	82880	Abu Dhabi
AF	AFG	Afghanistan	AFN	AS	29121286	647500	Kabul
AG	ATG	Antigua and Barbuda	XCD	NA	86754	443	St. John's
AI	AIA	Anguilla	XCD	NA	13254	102	The Valley
AL	ALB	Albania	ALL	EU	2986952	28748	Tirana

Така залепваме 3 таблици една за друга, макар, че средната няма да се вижда, т.к. не сме изброили нейна колона.


```
-- Коя река през коя държава минава
SELECT r.river_name, r.length, c.country_name
FROM rivers as r
JOIN
countries_rivers AS cr
ON r.id = cr.river_id връзка id1 = id2
JOIN
countries AS c
ON cr.country_code = c.country_code; връзка id2 = id3
```

Резултат – една река минава през много държави, една държава има много реки:

river_name	length	country_name
Nile	6650	Burundi
Nile	6650	Democratic Republic of the Congo
Nile	6650	Egypt
Nile	6650	Eritrea
Nile	6650	Ethiopia
Nile	6650	Kenya
Nile	6650	Rwanda
Nile	6650	Sudan
Nile	6650	South Sudan
Nile	6650	Tanzania
Nile	6650	Uganda
Amazon	6400	Bolivia
Amazon	6400	Brazil
Amazon	6400	Colombia
Amazon	6400	Ecuador
Amazon	6400	Guyana

✓ 186 11:58:56 SELECT r.river_name, r.length, c.country_name FROM rivers as r JOIN countries_rivers A... 104 row(s) returned

Този изглед е много неудобен. Много по-удобно би било да се напише името на една река и за нея да се изредят всички държави, през които тя минава. Това може да се осъществи с групиране – името на реката да е едно, а държавите да се изброят:

```
-- Коя река през кои държави минава, групирани
SELECT r.river_name AS River, c.country_name AS Countries
FROM rivers as r
JOIN
countries_rivers AS cr
ON r.id = cr.river_id
JOIN
countries AS c
ON cr.country_code = c.country_code
GROUP BY r.id; условие за групиране
```

Като прочетем грешката, тя ни подсказва, че липсва групиращо условие в SELECT за имената на държавите.

✖ 318 19:01:37 SEL... Error Code: 1055. Expression #2 of 'SELECT list is not in GROUP BY clause and contains nonaggregated column 'geography.c.country_name''

Това може да се осъществи с групово конкатениране:

```
-- Коя река през кои държави минава, групирани
SELECT r.river_name AS River, GROUP_CONCAT(' ', c.country_name) AS Countries
FROM rivers AS r
JOIN
countries_rivers AS cr
ON r.id = cr.river_id
JOIN
countries AS c
ON cr.country_code = c.country_code
GROUP BY r.id;
```

изреждане в група - държави

критерий за група - река

River	Countries
Nile	Burundi, Democratic Republic of the Congo, Eg...
Amazon	Bolivia, Brazil, Colombia, Ecuador, Guyana, Per...
Yangtze	China
Mississippi	Canada, United States
Yenisei	Mongolia, Russia
Yellow River	China
Ob	China, Kazakhstan, Mongolia, Russia
Paraná	Argentina, Bolivia, Brazil, Paraguay, Uruguay
Congo	Angola, Burundi, Democratic Republic of the Co...
Amur	China, Mongolia, Russia
Lena	Russia
Mekong	China, Cambodia, Laos, Myanmar, Thailand, Vi...

✔ 314 18:57:11 SELECT r.river_name AS River, GROUP_CONCAT(' ', c.country_name) AS Countries FR... 30 row(s) returned

Имаме групиране и изреждане, разделено със запетай, а броят на резултата е с 30 реда за реки, вместо 104. Можем да проверим дали е верен:

```
60 • SELECT COUNT(*) FROM rivers;
```

Result Grid

COUNT(*)
30

Да, отговорът е достоверен.

Винаги можем да добавим условие с WHERE, например, за да намерим всички реки, които минават през България:

```
-- Всички реки, които минават през България:
SELECT r.river_name, r.length, c.country_name
FROM rivers as r
JOIN
countries_rivers AS cr
ON r.id = cr.river_id
JOIN
countries AS c
ON cr.country_code = c.country_code
WHERE c.country_code='BG';
```

river_name	length	country_name
Danube	2888	Bulgaria

✓ 187 12:12:26 SELECT r.river_name, r.length, c.country_name FROM rivers as r JOIN countries_rivers A... 1 row(s) returned

Аналогично, най-после можем да изброим и всички книги и автори от БД за библиотека, която правехме в началото:

```
-- books = a, books_authors = b, authors = c
SELECT a.title,
       GROUP_CONCAT(c.first_name, ' ', c.last_name ORDER BY c.first_name) author
FROM   books a
       INNER JOIN books_authors b
           ON a.id_book = b.id_Book
       INNER JOIN authors c
           ON b.id_author = c.id_author
GROUP BY a.id_book, a.title;
```

Резултатът е 13 реда, които са групирани по име на книга и заглавие:

title	author
Под игото	Иван Вазов
Крадецът на праскови	Емилиян Станев
Под води и гори	Емилиян Станев
Изворът на белоногата	Петко Р. Славейков
Епически песни	Пенчо Славейков
Сън за щастие	Пенчо Славейков
Стихотворения	Димчо Дебелянов
Готварската книга на Дядо Славейков	Пенчо Славейков
Винету	Карл Май
Горски призрак	Карл Май
Коли	Стивън Кинг
Добри поличби	Нийл Геймън, Тери Пратчет
Песен за огън и лед	Джордж Мартин

✓ 255 12:24:16 SELECT a.id_book, a.title, GROUP_CONCAT(c.first_name, ' ', c.last_name O... 13 row(s) returned

Задача: Избройте върховете, в кои планини се намират и в кои държави.

Задача – Да се изброят държавите, в които няма планини.

Решение: Държавите са 250, а планините много по-малко. Т.к. ще преброяваме държавите, таблица countries е най-важна, тя ще бъде лявата LEFT JOIN, ще я вземем цялата за преброяването. Другата таблица е mountain_countries, която, както виждаме, има много пропуснати номера за номера на планини в държави:

mountain_id	country_code
3	AR
4	BG
16	BG
17	BG
23	BG
24	BG
25	BG
18	CA
26	CH
3	CL
9	CN

Всъщност, таблицата е с едва 29 реда записи:

✓ 333 20:21:58 SEL... 29 row(s) returned

От пръв поглед виждаме дори, че номер 7 липсва.

Result Grid		Filter Rows: 7
	mountain_id	country_code
▶	17	BG
*	NULL	NULL

Т.е. за условието държави, в които няма планини, не трябва да съществува mountain_id.

```
-- Държавите, в които няма планини
SELECT COUNT(*) AS country_count
FROM countries AS c
LEFT JOIN mountains_countries AS mc
    ON c.country_code = mc.country_code
WHERE mc.mountain_id IS NULL;
```

Резултат:

country_count
231

Последно: Когато се използва GROUP BY, след него, вместо WHERE се използва **HAVING** за задаване на условие.

преди GROUP BY – се използва WHERE

след GROUP BY – се използва HAVING.